zeroriscy_load_store_unit

OVERVIEW

The Load Store Unit (LSU) is responsible for handling all memory read and write operations in the processor. It acts as the interface between the Execution (EX) stage and the data memory system, ensuring correct and efficient data transfer during load and store instructions.

The LSU manages memory transactions by handling address alignment, byte enable generation, data rotation, sign extension, and misaligned memory accesses. It also supports request-grant-response handshaking with memory to ensure reliable communication.

This module is essential for maintaining correct memory access behavior while ensuring smooth integration with the processor pipeline.

FUNCTIONAL DESCRIPTION: CORE RESPONSIBILITIES

1. Memory Access Handling

The LSU generates and manages all memory access requests. Its responsibilities include:

- Generating memory request signal data_req_o
- Waiting for memory grant data_gnt_i
- Waiting for valid response data_rvalid_i

This ensures safe and synchronized communication with the memory subsystem.

2. Address and Data Alignment

The LSU ensures that data is correctly aligned before memory access. It performs:

- Write data alignment based on address offset
- Extraction of correct bytes, half-words, or words from read data

This guarantees accurate memory operations for all data types.

FUNCTIONAL DESCRIPTION: EXTENDED RESPONSIBILITIES

3. Byte Enable Generation

The module generates data_be_o (byte enable signal) depending on data type (word, half-word, byte), address alignment, and misalignment conditions. This ensures that only the required memory bytes are accessed.

4. Sign Extension

The LSU supports both signed and unsigned load operations. It performs byte to 32-bit extension and half-word to 32-bit extension, ensuring correct handling of signed and unsigned values.

5. Misaligned Access Handling

The LSU detects memory accesses that are not properly aligned (e.g., Word access not aligned to 4 bytes, or Half-word crossing address boundary). In such cases, the LSU:

- Splits the access into two memory transactions
- Stores partial data temporarily
- Reassembles the final result correctly

6. Pipeline Control

The LSU generates control signals for pipeline synchronization:

- busy_o -> Indicates LSU is active
- data_valid_o -> Indicates valid data is ready for the register file

KEY FEATURES

- Supports byte, half-word, and word accesses
- Handles both aligned and misaligned memory operations
- Implements write data rotation for alignment
- Provides sign and zero extension
- Uses FSM-based control logic
- Compatible with handshake-based memory interfaces
- Error outputs currently hardwired (load_err_o = 0, store_err_o = 0)

INPUT AND OUTPUT SIGNALS: CLOCK & MEMORY INTERFACE

Clock and Reset:

- clk -> System clock
- rst_n -> Active-low reset

Outputs to Memory:

- data_req_o -> Memory request signal
- data_addr_o [31:0] -> Memory address
- data_we_o -> Write enable signal
- data_be_o [3:0] -> Byte enable signal
- data_wdata_o [31:0] -> Write data sent to memory

Inputs from Memory:

- data_gnt_i -> Grant signal from memory
- data_rvalid_i -> Read data valid signal
- data_rdata_i [31:0] -> Read data returned from memory
- data_err_i -> Error signal

INPUT AND OUTPUT SIGNALS: EX STAGE & PIPELINE

Execution Stage Interface:

- data_req_ex_i -> Request from EX stage
- data_we_ex_i -> Write enable from EX stage
- data_type_ex_i [1:0] -> Data type selector (00: Word, 01: Half-word, 10/11: Byte)
- data_wdata_ex_i [31:0] -> Write data from EX stage
- data_reg_offset_ex_i [1:0] -> Register offset
- data_sign_ext_ex_i -> Sign extension control
- adder_result_ex_i [31:0] -> Calculated memory address

Outputs to EX Stage / Pipeline:

- data_rdata_ex_o [31:0] -> Read data output
- data_valid_o -> Indicates data is ready
- busy_o -> LSU busy signal

Misalignment and Exception Signals:
- data_misaligned_o -> Indicates misaligned access
- misaligned_addr_o [31:0] -> Address for second memory access
- load_err_o -> Load error signal (currently unused)
- store_err_o -> Store error signal (currently unused)

OPERATION FLOW

Load Operation:
1. EX stage sends request using data_req_ex_i
2. LSU generates memory request and waits for Grant and Data valid
3. Correct portion of read data is extracted and sign extension is applied
4. Final data is sent to the register file

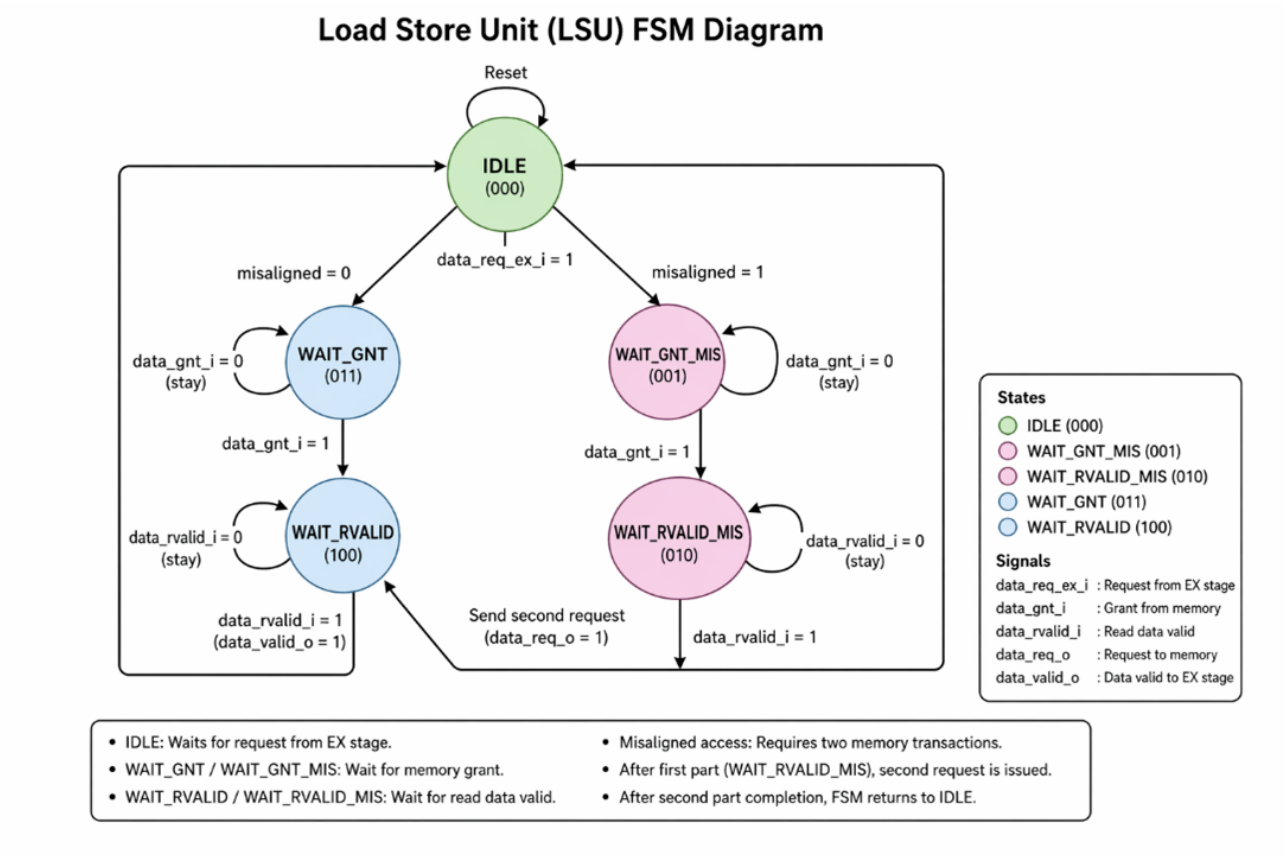
Store Operation:
1. EX stage provides write data
2. LSU performs data alignment using rotation and generates byte enables
3. Aligned data is sent to memory and completes after memory grant

Misaligned Access Handling:
Detected when a word is not aligned to a 4-byte boundary or a half-word crosses an address boundary. The LSU performs the first memory access, stores partial data, performs the second memory access, and combines both results correctly.

FINITE STATE MACHINE (FSM)

The LSU uses an FSM to manage memory transactions efficiently.
FSM States:
- IDLE: Waiting for request
- WAIT_GNT: Waiting for memory grant
- WAIT_RVALID: Waiting for read data
- WAIT_GNT_MIS: Waiting for grant (misaligned access)
- WAIT_RVALID_MIS: Waiting for read data (misaligned access)

FSM DIAGRAM



FSM BEHAVIOR SUMMARY

- IDLE -> WAIT_GNT / WAIT_GNT_MIS (New memory request arrives)
- WAIT_GNT -> WAIT_RVALID (Memory grant is received)
- WAIT_RVALID -> IDLE (Valid read data is received)

Misaligned operations require two memory cycles and intermediate buffering using rdata_q to correctly reconstruct split memory accesses.

IMPORTANT INTERNAL MECHANISMS

Byte Enable Logic:
Determines which bytes are accessed in memory depending on the lower address bits and data type.

Write Data Rotation:
Used to align register write data with memory address boundaries using circular shifting. This is especially important for byte and half-word stores.

Read Data Assembly:
For misaligned reads, the LSU uses rdata_q for temporary storage to combine partial data from multiple memory accesses.

Sign Extension Logic:
Controlled by data_sign_ext_ex_i, supporting both signed and unsigned load operations to ensure proper

interpretation of loaded values.

SUMMARY

The zeroriscy_load_store_unit is a critical memory interface module responsible for reliable and efficient data transfer between the processor and memory.
It ensures:

- Correct handling of load and store instructions
- Proper alignment of addresses and data
- Robust support for misaligned memory accesses
- Accurate sign and zero extension
- Seamless coordination with pipeline stages

Using an FSM-driven architecture, the LSU manages memory transactions safely and efficiently, making it one of the most important modules in the Zero-Riscy processor pipeline.